

Synapse Architect MasterCourse

Complete 8-Module Architect Textbook

Author: ChatGPT AI

Table of Contents

1. Module 1 – Internal Architecture & Engine Design
2. Module 2 – Advanced ADLS & Lakehouse Engineering
3. Module 3 – Serverless SQL Internals & Optimization
4. Module 4 – Dedicated SQL MPP Internals
5. Module 5 – Advanced Spark & Delta Lake
6. Module 6 – Pipelines, Metadata Frameworks & CI/CD
7. Module 7 – Security, Networking & Governance
8. Module 8 – Troubleshooting & Diagnostics

Synapse Architect MasterCourse – Module 1

Internals & Core Engine Architecture (Expanded Edition)

1. Overview of Synapse Internal Architecture

Azure Synapse Analytics is designed around a decoupled architecture where compute and storage are isolated.

This enables scaling both independently while also allowing multiple compute engines such as Serverless SQL, Dedicated SQL, Spark, and Data Explorer to read and write from the same underlying data lake (ADLS Gen2).

A Synapse Architect must understand:

- The flow of data inside Synapse.
- How each compute engine interacts with ADLS.
- Internal components that handle orchestration, data movement, caching, metadata, and security.

2. Distributed MPP Architecture of Dedicated SQL Pool

Dedicated SQL Pool uses an MPP system with the following components:

Control Node:

- Receives T-SQL queries.
- Generates distributed execution plans.
- Coordinates compute nodes.

Compute Nodes:

- Execute operations over data slices.
- Hold part of the distributed columnstore segments.

Distributions (60 total):

- Smallest parallel processing units.
- Data distributed using HASH, ROUND_ROBIN, or REPLICATE.

DMS (Data Movement Service):

- Moves data across nodes during JOIN, GROUP BY, or shuffle operations.

3. Distribution Strategies

HASH Distribution:

- Best for large fact tables.
- Requires high-cardinality column.
- Minimizes shuffles if join keys align.

ROUND_ROBIN:

- Used for staging tables.
- Not optimal for analytical joins.

REPLICATE:

- Duplicates entire table across all nodes.
- Useful for small dimensions (<2GB).

4. Serverless SQL Engine Internals

Serverless SQL performs:

1. Metadata resolution
2. Filter pushdown analysis
3. Partition pruning
4. Column projection
5. Row-group pruning (Parquet optimized)
6. Parallel file scanning
7. Result reconstruction

Serverless SQL charges per TB scanned, hence data pruning is essential.

5. Spark Engine Internals

Synapse Spark is built on Apache Spark with:

- Catalyst Optimizer (logical optimization)
- Tungsten Engine (physical optimization)
- DAG Scheduler (stage execution)

Performance depends on:

- Partition sizes
- Shuffle volume
- Use of vectorized IO (Parquet/Delta)
- Avoiding Python UDFs

6. Caching & Memory Internals

Synapse includes multiple caching layers:

Dedicated SQL Cache:

- Caches compressed columnstore segments.
- Cleared on pause or scale.

Serverless SQL Cache:

- Caches results for 30 minutes.
- Reduces repeated query cost.

Spark Cache:

- Memory + spill-to-disk.
- Evicted when Spark session ends.

Understanding cache behavior is essential to optimizing workloads.

Example: Serverless SQL Query with Pushdown

```
SELECT * FROM OPENROWSET(  
  BULK 'https://storage/data/year=2025/month=01/*.parquet',  
  FORMAT='PARQUET'  
) WITH (id INT, ts DATETIME) AS rows  
WHERE year=2025 AND month=1;
```

Synapse Architect MasterCourse – Module 2

Advanced ADLS Engineering & Lakehouse Data Design (Expanded Edition)

1. ADLS Gen2 Architecture Internals

Azure Data Lake Storage Gen2 is the foundation of the Synapse Lakehouse. It combines hierarchical namespace capabilities with high-throughput blob storage.

Key internal components:

- Blob containers → logical boundaries
- File system API → POSIX-like operations
- Access control via RBAC + ACL
- Optimized metadata store

2. Enterprise Folder Structure Design

A well-designed folder layout improves scalability, performance, lineage, and governance. The recommended layout is:

```
/raw/////
/silver///delta_logs/
/gold///optimized/
```

Raw → No schema enforcement
Silver → Cleaned, deduped, validated
Gold → Business-ready, aggregated

3. ADLS Performance Considerations

Challenges in enterprise lakes:

- Millions of small files slow down listing operations.
- Spark jobs slow when scanning directories with too many partitions.
- ACL evaluations slow on deep directory structures.

Solutions:

- Use Delta Lake to reduce file counts via OPTIMIZE.
- Use coalesced Parquet writes.
- Keep domain-level folder structures shallow.

4. Handling Schema Drift

Schema drift occurs when incoming files have different columns.

Synapse Lakehouse strategy:

- Bronze: allow drift entirely
- Silver: enforce schema using Spark
- Gold: maintain strict schema models

This ensures flexibility at ingestion and stability at consumption.

5. Multi-Tier Storage Strategy

Use lifecycle management to minimize cost:

Hot tier:

- Silver tables (frequent use)

Cool tier:

- Raw data older than 30 days

Archive tier:

- Long-term retention

Azure policies help automate these transitions.

6. Metadata Optimization

Metadata operations (like LIST and GETATTR) become slow at scale.

Best practices:

- Avoid >10k files per partition.
- Avoid deeply nested folders beyond 6 levels.
- Use partition pruning with year/month/day.
- Always write Parquet/Delta instead of CSV.

Synapse Architect MasterCourse – Module 3

Advanced Serverless SQL & Performance Optimization (Expanded Edition)

1. Serverless SQL Engine Deep Internals

Serverless SQL is a distributed, on-demand compute engine that pushes most processing to the storage layer.

The query engine optimizes cost by reducing the bytes scanned. Key internal phases include:

- Metadata resolution
- Predicate pushdown
- File-level pruning
- Row-group (Parquet) pruning
- Column projection
- Distributed parallel scanning
- Adaptive result merging

2. Performance Optimization Techniques

Serverless SQL charges based on TB scanned. To optimize:

- Use Parquet over CSV (90% reduction)
- Avoid SELECT *
- Filter early on partition columns
- Store data as Delta/Parquet
- Avoid nested folders where partitioning is expected
- Explicit schema definitions via OPENROWSET WITH

3. Partition Pruning Internals

Partition pruning allows serverless SQL to skip entire folder paths based on WHERE predicates.

Example folder structure:

/data/year=2024/month=01/day=12/*.parquet

Query filters must reference partition columns directly:

WHERE year=2024 AND month=1 AND day=12

Avoid:

WHERE CAST(datecol AS DATE) = '2024-01-12'

4. Best Practices for Serverless SQL External Tables

External tables are useful for BI workloads and provide metadata definitions for file-based tables.

Best practices:

- Define WITH schema explicitly
- Use proper folder partitioning
- Avoid wildcards in root-level paths
- Use CETAS to generate optimized output

5. Cost Management Strategies

Serverless SQL cost is predictable when you:

- Partition properly
- Materialize heavy queries into CETAS
- Avoid reading entire folders
- Tune schema and file sizes (256MB–1GB)
- Avoid reading JSON/CSV directly

6. Example Queries

OPENROWSET Parquet Query:

```
SELECT *  
FROM OPENROWSET(  
  BULK 'https://lake/data/year=2024/month=01/*.parquet',  
  FORMAT='PARQUET'  
) WITH (id INT, value FLOAT, ts DATETIME) AS rows  
WHERE year=2024 AND month=1;
```

Synapse Architect MasterCourse – Module 4

Dedicated SQL Pool Internals & MPP Optimization (Expanded Edition)

1. MPP Architecture Deep-Dive

Dedicated SQL Pool is an MPP system with:

- Control Node for planning and orchestration.
- Compute Nodes running distributed queries.
- 60 Distributions executing tasks in parallel.
- Data Movement Service (DMS) for shuffles.

Understanding these components is essential for architect-level performance tuning.

2. Columnstore Index Internals

Columnstore indexes compress data into column segments and row groups.

Performance depends on:

- Segment elimination
- Proper ordering
- Statistics distribution

3. Distribution Optimization

Choose distribution keys based on:

- High cardinality
- Even distribution
- Common join keys

4. CTAS & ELT Strategy

CTAS (CREATE TABLE AS SELECT) is the fastest transformation tool.

It:

- Runs in parallel
- Writes optimal columnstore segments
- Avoids transaction log overhead

5. COPY INTO Optimization

Use COPY INTO for bulk ingestion from ADLS.

Best practices:

- Load Parquet over CSV
- Use 256MB–1GB file sizes
- Parallelize load with multiple files

6. Resource Classes

Resource classes define memory allocation.

Larger classes improve join performance but reduce concurrency.

Synapse Architect MasterCourse – Module 5

Advanced Spark & Delta Lake Engineering (Expanded Edition)

1. Spark Execution Internals

Spark operates with:

- Driver managing the DAG
- Executors running distributed tasks
- Catalyst Optimizer for logical plans
- Tungsten engine for optimized execution

The performance depends on partitioning, shuffle operations, caching, and the use of built-in functions instead of UDFs.

2. Delta Lake Internals

Delta Lake adds ACID transactions to Parquet files using:

- Delta Log (JSON + Parquet checkpoint)
- Atomic operations
- Snapshot isolation
- Schema evolution or enforcement

Delta enables reliable pipelines, time travel, and optimized merging.

3. Partitioning Strategy

Partitioning large tables is essential:

- Use high-selectivity columns
- Avoid over-partitioning small tables
- Keep partitions between 128MB–1GB

Partition pruning dramatically reduces scan cost.

4. OPTIMIZE & ZORDER

OPTIMIZE compacts many small files into larger files.

ZORDER improves data skipping for high-cardinality columns.

Use ZORDER for:

- JOIN keys
- Timestamp columns
- High-cardinality filters

5. Streaming Architecture

Structured Streaming supports:

- Micro-batch processing
- Exactly-once sinks
- Auto checkpointing
- Watermarking

Synapse Spark allows streaming → Silver → Gold transformation pipelines.

6. Advanced Delta Operations

MERGE INTO allows fast upserts.

UPDATE/DELETE operations depend on Delta Log internals.

VACUUM ensures old files are cleaned after retention period.

Synapse Architect MasterCourse – Module 6

Pipelines, Metadata Framework, & CI/CD (Expanded Edition)

1. Synapse Pipelines Architecture

Synapse Pipelines use the same runtime as Azure Data Factory (ADF). They support ETL/ELT workflows, orchestration, data movement, and metadata-driven automation.

Core components:

- Pipelines
- Activities (Copy, Notebook, SQL, Web, Data Flows)
- Integration Runtimes

2. Types of Integration Runtimes

AutoResolve IR:

- Default
- Best for cloud-to-cloud movement

Azure IR:

- Supports high-throughput parallelism
- Data Flow executions

Self-Hosted IR:

- Required for on-premises data sources
- Supports hybrid connectivity

3. Metadata-Driven Framework Design

Enterprise ETL systems require metadata-based execution.

Key design patterns:

- Parameterized pipelines
- Lookup tables for dynamic source-to-target mapping
- Generic ingestion templates
- Audit tables for pipeline events

This avoids building hundreds of pipelines manually.

4. Data Flows – Internal Mechanics

Mapping Data Flows run on Spark clusters internally.

Features include:

- Code-free transformations
- Schema drift handling
- Automatically scaled compute
- Partition awareness

Best used for transformations where SQL/Spark notebooks are not preferred.

5. Orchestration Patterns for Lakehouse

Typical orchestrations:

- Raw ingestion → Bronze Delta
- Validation and cleansing → Silver Delta
- Aggregation and curation → Gold Delta
- Refreshing Power BI datasets
- Data quality rule checks

6. CI/CD in Synapse

Synapse integrates with Git to support collaboration and version control.

CI/CD workflow:

1. Connect workspace to Git
2. Develop in feature branches
3. Publish to workspace
4. Deploy using ARM templates / Synapse artifacts
5. Promote across DEV → TEST → PROD

Each environment should have:

- Separate workspace
- Separate ADLS containers
- Parametrized linked services

Synapse Architect MasterCourse – Module 7

Security, Networking & Governance (Expanded Edition)

1. Synapse Security Architecture

Synapse uses a multi-layer security model:

- Azure RBAC for workspace access.
- Synapse RBAC for SQL/Spark permissions.
- SQL authentication & authorization.
- ADLS ACLs for fine-grained file access.

A strong architecture uses all four layers harmoniously.

2. Managed Virtual Network

Managed VNET isolates Synapse resources inside a secure Microsoft-managed network.

Key capabilities:

- No public IP exposure.
- Managed private endpoints to storage, SQL, and Key Vault.
- Data exfiltration protection.

This is mandatory for regulated industries.

3. Private Endpoints

Synapse requires private endpoints for:

- SQL endpoint
- Dev/Prod workspace access
- ADLS storage account
- Key Vault
- Data Explorer pool

These endpoints enforce locked-down, internal-only traffic.

4. Key Vault & CMK Encryption

Synapse integrates with Azure Key Vault for:

- Secret management
- Credential storage
- Customer Managed Keys (CMK)

CMK is used for encrypting dedicated SQL pools and lake data.

5. Governance with Purview

Purview provides:

- Data cataloging
- Classification (PII, sensitivity labeling)
- Lineage tracking
- Automated scanning for Synapse SQL, Spark, Pipelines, ADLS

Essential for enterprise data governance.

6. Zero-Trust Architecture

Zero Trust includes:

- Explicit identity validation
- MFA & Conditional Access
- Role separation
- No publicly accessible endpoints
- Network segmentation
- Just-in-time access

Synapse's managed VNET + private endpoints model supports Zero Trust.

Synapse Architect MasterCourse – Module 8

Troubleshooting, Diagnostics & Architecture Patterns (Expanded Edition)

1. Dedicated SQL Pool Troubleshooting

Key DMV queries:

- sys.dm_pdw_exec_requests → query history
- sys.dm_pdw_dms_workers → shuffle diagnostics
- sys.dm_pdw_request_steps → execution steps

Common issues:

- Data skew
- Shuffle bottlenecks
- Spill-to-disk
- Statistics out of date

2. Spark Troubleshooting Techniques

Monitor:

- DAG visualization
- Stage duration
- Task skew
- Shuffle size
- Memory pressure (OOM, GC)

Optimization:

- Replace UDFs with built-in functions
- Repartition intelligently
- Use broadcast joins for small lookup tables

3. Serverless SQL Diagnostics

Common errors:

- Excessive IO (too many TB scanned)
- Schema drift failures
- Unsupported Parquet metadata types

Fixes:

- Use partition pruning

- Define schema explicitly
- Validate file structures

4. Pipeline Troubleshooting

Common failures:

- IR connection timeout
- Credential failures
- Copy activity throughput issues

Solutions:

- Use parallel copy units
- Validate IR network settings
- Use optimized file formats

Logging & monitoring:

- Activity run logs
- Integration runtime diagnostics

5. Advanced Monitoring Tools

Tools available:

- Log Analytics
- Azure Monitor
- Synapse Workspace Monitoring
- Spark UI
- SQL analytics in Azure Portal

Monitor:

- CPU/memory usage
- Pipeline failures
- Storage throughput
- Real-time metrics

6. Architecture Patterns & Anti-Patterns

Patterns:

- Medallion Lakehouse (Bronze/Silver/Gold)
- SQL Marts fed by Gold layer
- Delta + Spark transformations
- Serverless BI exploration layer

Anti-patterns:

- Storing CSV as final layer

- Lack of partitioning
- Using ROUND_ROBIN for fact tables
- Building pipelines without metadata framework